Available online at www.sciencedirect.com
ScienceDirect
Fuzzy Sets and Systems 159 (2008) 1577–1596
FUZZY sets and systems
www.elsevier.com/locate/fss

GDB: A tool to build deductive rules using a fuzzy relational database with scientific data

The International GIADA Team, IDBIS Research Group

R. Morales[a,*], I. Blanco[b], O. Pons[b], J. Rodríguez[a]

[a]*Instituto de Astrofísica de Andalucía, CSIC, Granada, Spain*
[b]*Departmento Ciencias de la Computación e I.A. Universidad de Granada, Spain*

Available online 3 February 2008

Abstract

This paper presents GDB, a system to build, debug and test rules in a Fuzzy Relational Deductive Database System (FRDDS) in a visual way avoiding the associated syntax which could be quite complex for a non-expert user. The rules will be used to compute a Measure of Quality (MoQ) of the scientific data generated by the GIADA instrument (ROSETTA space mission). The MoQ will be defined and assigned taking into account the knowledge of the engineers who built the instrument and the formal documentation. Together with this MoQ, the tool will provide a semantic explanation about how the measure has been assigned. With GDB is possible to work with imprecise data, to store knowledge using rules and also to deduce new information. The MoQ and the explanation will provide to the scientists and engineers a better understanding about the instrument behavior and, therefore, about physical phenomena under study.
© 2008 Published by Elsevier B.V.

Keywords: Fuzzy databases; Reliability; GIADA; Deductive rule; Fuzzy deductive database; Data quality

1. Introduction

Data generated by a scientific instrument do not always match the scientists/engineers assumptions about phenomena under study. In these cases, it is necessary to discover where the misunderstanding is: in the data quality (e.g. the data must be ignored) or in the assumption. In this paper we focus on building a tool that helps to solve the first problem for the GIADA instrument. The tool calculates and assigns a Measure of Quality and an Explanation (MoQ + E) to the acquired data. The MoQ indicates the data reliability and is assigned using the knowledge about the instrument. The Explanation is a set of references showing how a concrete MoQ is assigned. With the Explanation it is possible to check all the steps followed by the tool to assign the MoQ using a human readable format.

Usually, the only way to assign a data quality is to interpret the own data studying the instrument manuals and the technical documentation. Frequently it is easy to put all this knowledge in rule format: *if*...*then* and then use certain type of tools to manage them. *Logic* [23] is one of the tools that allow the deduction processes (can generate information not stored implicitly in the input rule set) to be carried out.

* Corresponding author.
E-mail address: rmorales@iaa.es (R. Morales).

0165-0114/$ - see front matter © 2008 Published by Elsevier B.V.
doi:10.1016/j.fss.2008.01.022

From the traditional point of view, scientific instruments generate *precise* data collected from physical sensors (temperature, voltage...). But these sensors are intrinsically not precise; there is an error margin that must be taken into account. This margin is increased gradually with the instrument use and the instrument life time. So, the instrument precise data derive to imprecise data in a simple way and this knowledge must be used to obtain valid sensor values along the instrument life.

The proposed system (called GDB:**G**IADA **D**ata**B**ase) will use rules to store the instrument knowledge and imprecise data to capture the sensor evolution. All this information will be stored in a Relational Database Management System (RDBMS) that can support logic rules and fuzzy data.

The paper is organized as follows: in Section 2, we introduce the previous models, architectures and implementations, the Logical Model (classical and fuzzy) for relational databases and the deduction algorithm used in GDB. The modifications performed to the previous works to meet the GDB objectives are detailed in Section 3. Section 4 is devoted to describe the GIADA instrument, the GDB characteristics and an example of use. Finally, in Section 5 are outlined the conclusions and future research lines.

2. Previous models

Traditionally, the Relational Database Model (RDBM) [10] has been used as powerful tool for representation and data handling. But this model cannot manage the data imprecision and only generates stored explicitly data (it cannot deduce new information). Various models have been proposed to extend the RDBM in order to solve these limitations.

2.1. GEFRED

Historically, the Fuzzy Set Theory [30] has been used as main tool for the storage and manipulation of fuzzy data. This theory is the base layer of the RDBM extensions for representing and handling fuzzy information. These approaches are divided in two main trends: unification models by similarity relations [7,27] and possibilistic models (mainly [26,28,31]. GEFRED [19] is a theoretical model that synthesizes the more outstanding aspects of both approaches.

2.1.1. Data structure in GEFRED

The fuzzy data is organized via the following elements:

Definition 1. *Generalized fuzzy domain* (GFD) is a set that verifies:

$$D_{\mathrm{G}} \subseteq \tilde{\mathcal{P}}(D) \cup \{\mathrm{NULL}\},$$

where D is a discourse domain, $\tilde{\mathcal{P}}(D)$ is the set of all possibility distributions defined on D (including those defining values UNKNOWN and UNDEFINED). A GFD can contain values of one of the following types.

Definition 2. *Generalized fuzzy relation* (GFR) is a pair of sets $(\mathcal{H}, \mathcal{B})$ where:

- $\mathcal{H}$ (named head) is the set of triplets {attribute, GFD, compability degree} that defines the structure of the relation. The last triplet element is optional.

$$H = \{(A_{G_1} : D_{G_1}[, C_{A_{G_1}}]), \ldots, (A_{G_n} : D_{G_n}[, C_{A_{G_n}}])\}.$$

- $\mathcal{B}$ (named body) is the set of triplets {attribute, value, degree} that defines the content of the relation. The last triplet element is optional.

$$B = \{\{(A_{G_1} : \tilde{d}_{i1}[, C_{i1}]), \ldots, (A_{G_n} : \tilde{d}_{in}[, C_{in}])\}.$$

We can structure the data through a relation model, R_{FG}, given by

$$R_{FG} \subset (D_{G1}, C_1), \times, \ldots, \times(D_{Gn}, C_n),$$

where every D_{Gj} is a GFD, and C_j is a "compatibility attribute'' that takes its values from [0,1]. C_j represents the fulfilment degree respect to a condition of a domain value D_{Gj}. This relation represents the initial information as well

as that resulting from the Fuzzy Algebra operations. Handling these relations through Fuzzy Relational Algebra can modify, for every tuple, the compatibility attribute values. Classical relations are a particular case of R_{FG} where the compatibility attribute is always equal to one.

2.1.2. Data manipulation in GEFRED

In order to compare values in the same GFD, the Generalized Fuzzy Comparator definition based on the Extended Fuzzy Comparator is introduced.

Definition 3. *Extended fuzzy comparator* (EFC) θ is a function defined on the discourse domain D as

$$\theta : D \times D \to [0, 1],$$

$$\theta(d_i, d_j) \to a | a \in [0, 1].$$

Definition 4. *Generalized fuzzy comparator* (GFC) Θ^θ is a function defined on the discourse domain D as

$$\Theta^\theta : D_G \times D_G \to [0, 1],$$

$$\Theta^\theta(\tilde{d}_1, \tilde{d}_2) \to [0, 1],$$

verifying

$$\Theta^\theta(\tilde{d}_1, \tilde{d}_2) = \theta(d_1, d_2), \forall d_1, d_2 \in D,$$

where D_G is GFD defined on D, θ is an EFC defined on D, and $\tilde{d}_1, \tilde{d}_2$ are the possibility distributions $\{1/d_1\}$ and $\{1/d_2\}$ defined on the domain values d_1 and d_2, respectively.

In order to manipulate data (fuzzy or not), a Fuzzy Algebra is defined as an extension of the classical Relational Algebra [14]. Particularly, classical comparison operators are modified for handling the imprecise information. Also, typical algebraic operations and compound conditions (simple conditions combined using logic connectives: negation, conjunction and disjunction) are extended to calculate accordingly the compatibility degree.

2.2. FIRST

FIRST [16] is an implementation of the GEFRED model using the RDBMS Oracle®. Let us see some relevant aspects in the fuzzy data representation and handling.

2.2.1. Fuzzy data structure in FIRST

FIRST defines three types of table attributes that can be treated in an imprecise way. Note that FIRST supports all the data types defined for GEFRED (Table 1). All the extra information to represent the fuzzy data and its relations are stored in the Fuzzy Meta-Knowledge Base. The data type classification is based on the underlying domain.

- *Fuzzy Type* 1: Attributes with "crisp'' (not fuzzy) data that can have associated a linguistic label. Its representation is the same as used for the classical precise data. This attribute type allows fuzzy queries (using the linguistic label) on crisp data using triangles to represent approximate values (Fig. 1). The association between attribute and label is stored in the Fuzzy MKB (Fig. 2).
- *Fuzzy Type* 2: Attributes with "imprecise data on ordered domain''. The data types allowed and its table representation are described in Table 2.

Any *Fuzzy Type* 2 attribute is represented in a classical table (stored in the Fuzzy MKB) with an attribute F_TYPE with the type code and attributes F_1, F_2, F_3 and F_4 representing the parameters for each data. NULL values appearing in the attributes have the meaning assigned by the RDBMS. In the LABEL type the FUZZY_ID code represents an identifier for the linguistic label defined in the Fuzzy MKB base. Note that the fuzzy type (type 7) representation is really a trapezoid parameter definition.

Table 1
GEFRED data types

1	A single scalar (behavior = good, represented by the possibility distribution, 1/good)
2	A single number (age = 28, represented by the possibility distribution, 1/28)
3	A set of possible scalar assignments (behavior = {good,bad}, represented by {1/good, 1/bad})
4	A set of possible numeric assignments (age = {20, 21}, represented by {1/20, 1/21})
5	A possibility distribution in a scalar domain (behavior = {0.6/bad, 0.7/normal})
6	A possibility distribution in a numeric domain (age = {0../23, 0.8/25}, fuzzy numbers or linguistic labels)
7	A real number belonging to [0, 1], referring to degree of matching (quality = 0.9)
8	An **Unknown** value with possibility distribution, **Unknown** = $\{1/u : u \in U\}$
9	An **Undefined** value with possibility distribution, **Undefined** = $\{0/u : u \in U\}$
10	A **NULL** value given by **NULL** = {1/Unknown, 1/Undefined}

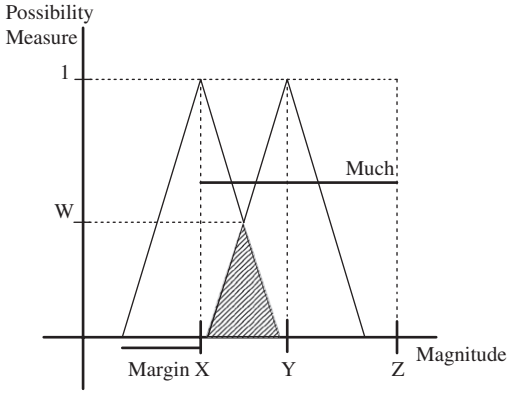

Fig. 1. Possibility measure calculation of two values.

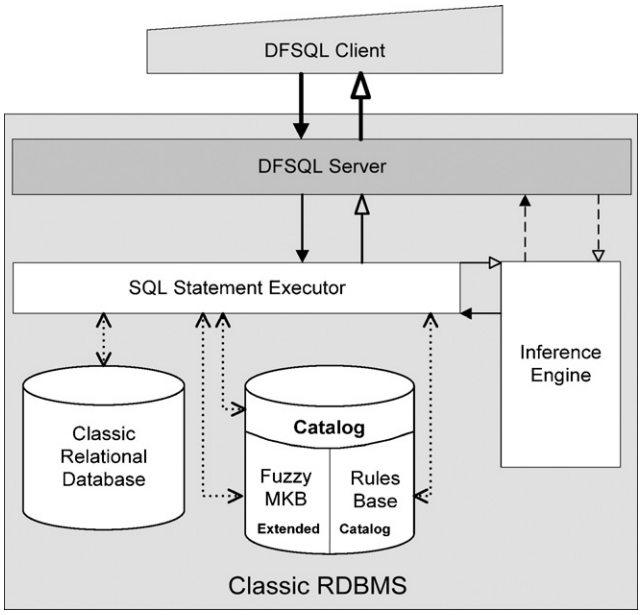

Fig. 2. FREDDI architecture with an integrated inference engine.

Table 2
Attributes representation for Fuzzy Type 2

Data type	F_TYPE	F_1	F_ 2	F_3	F_4
UNKNOWN	0	NULL	NULL	NULL	NULL
UNDEFINED	1	NULL	NULL	NULL	NULL
NULL	2	NULL	NULL	NULL	NULL
CRISP	3	d	NULL	NULL	NULL
LABEL	4	FUZZY_ID	NULL	NULL	NULL
INTERVAL [A,B]	5	A	0	0	B
APROX (d)	6	d-margin	Margin	−Margin	d + Margin
FUZZY	7	α	$\beta - \alpha$	$\gamma - \delta$	δ

Table 3
Attributes representation for Fuzzy Type 3

Data type	F_TYPE	F_P1	F_1	F_P2	F_2	...
UNKNOWN	0	NULL	NULL	NULL	NULL	...
UNDEFINED	1	NULL	NULL	NULL	NULL	...
NULL	2	NULL	NULL	NULL	NULL	...
SCALAR	3	1	d	NULL	NULL	...
POS. DISTR.	4	p_1	d_1	p_2	d_2	...

Table 4
Fuzzy operators in FIRST

Fuzzy possibility operators	
FEQ	Fuzzy EQual
FGT	Fuzzy Greater Than
FGEQ	Fuzzy Greater or Equal
FLT	Fuzzy Less Than
FLEQ	Fuzzy Less or Equal
MGT	Much Greater Than
MLT	Much Less Than
NFEQ	Necessarily Fuzzy EQual
Fuzzy necessity operators	
NFGT	Necessarily Fuzzy Greater Than
NFGEQ	Necessarily Fuzzy Greater or Equal
NFLT	Necessarily Fuzzy Less Than
NFLEQ	Necessarily Fuzzy Less or Equal
NMGT	Necessarily Much Greater Than
NMLT	Necessarily Much Less Than

- *Fuzzy Type* 3: Attributes with "not ordered discrete domains with analogies''. These attributes are used to represent scalar data or possibility distributions on scalar domains $\{p_1/d_1, \ldots, p_n/d_n\}$. The attribute type and its representation associated are stored in the Fuzzy MKB. The data types included in this fuzzy type are described in Table 3.

2.2.2. *Fuzzy operators in FIRST*

FIRST implements the classic binary comparators (=, >, ...) plus 14 GFCs [18] that can be applied on *Fuzzy Type* 1 or *Fuzzy Type* 2 attributes. The *FEQ* operator is the only operator that can be applied on *Fuzzy Type* 3 attributes (Table 4).

Fuzzy possibility operators are less restrictive than the Fuzzy necessity operators. The precise definition of these operators can be found in [16].

As an example, let us consider the application of a fuzzy operator *FEQ* on two *Fuzzy Data Type* 1 values: *X FEQ Y*.

W (the measure of the fuzzy equality) is defined as the intersection area of the two triangles defined around X and Y. Each triangle represents the possibility distribution associated with the value. The *margin* (half the triangle base of the possibility distribution associated to the value) indicates how large is the area around a value to be used in the fuzzy comparison. The *much* establishes the minimum distance between two values for the comparison makes sense. The values of *much* and *margin* must be defined in *Fuzzy Data Type* 1 or *Type* 2 declaration before applying the fuzzy operators. The comparison between *Fuzzy Data Type* 3 values has its own characterization measures (see [16]).

2.2.3. The FSQL language

In order to handle the fuzzy data types and operators previously defined, an extension of the traditional relational query language SQL is made. This extension is called Fuzzy SQL (FSQL) and includes a fuzzy select, that allows queries involving classical or fuzzy data [3,17].

2.3. FREDDI

FREDDI [20] establishes the architecture to represent rules and meta-knowledge using the commercial RDBMS Oracle®. The FREDDI components are:

- *DFSQL Client*: It accepts sentences (queries) in DFSQL language, send them to the DFSQL *Server* and receive the result tables.
- *DFSL Server*: It accepts sentences (queries) in DFSQL language from the clients and translates them into classical SQL sentences, executes them and returns the result tables. To carry out this task, it analyzes the input sentence and it separates the parts (clauses) that involve fuzzy or deductive processing. Then, collects data from the *Catalog*, the *Extended Catalog* and from the *Classical Relational Database*. To obtain these data it is necessary to build auxiliary SQL classical sentences and order its execution to the *SQL Statement Executor*. Then, all the information involved the deduction process is sent to the *Inference Engine* for obtaining a set of results. With these results and with the translation of the fuzzy clauses (into classical SQL sentences), the DFSQL Server builds a definitive SQL sentence. The final sentence is executed by the *SQL Statement Executor* and the result tables are processed before sending them back to the *DFSQL client*.
- *SQL Statement Executor*: It accepts classical SQL sentences and executes them into the Classical Relational Database. To perform that execution, it needs to access to the information included in the *Catalog* and also into the *Extended Catalog*. The relational classical tables obtained as result are sent back to the module that required its execution.
- *Classical Relational Database*: It stores all the data (fuzzy or not) into the classical relational tables.
- *Catalog*: It contains the meta-data or information about the stored classical data: structure, localization. . .
- *Extended Catalog*: It contains the meta-data or information about the stored fuzzy data and the stored rules. It is composed by the *Fuzzy MKB* that stores the fuzzy meta-data and the *Rules Base* that stores the logic meta-data.
- *Inference Engine*: It uses the stored logic rules to deduce new implicit data (not stored previously) using the stored data and the rules present in the *Rule Base*. To do it, it requires that the *SQL Statement Executor* run a set of classical SQL sentences.

2.4. Logical Model for Relational Databases

One of the most used tools for performing deductions and for representing the human knowledge if Logic [23]. In this section we review the classical approximation to use the Logic in a Relational Database and its extension to handle fuzzy data.

2.4.1. Logic Database Model

This first attempt to adapt the Logic into the RDBM was made by Gallaire [15]. The proposed Logic Database Model provides a translation for every element in the RDBM into the logic language and the necessary axioms for the correct and complete description of a Database. Also a logic programming language as Prolog [6] is adapted, developing a specific language for deduction in relational databases named Datalog [9]. According to this model, every predicate

corresponds to a relation (table) and facts correspond to tuples (table rows). A predicate is defined by means of a rule. The Logic Database Model uses the following Prolog-Type rule format:

Definition 5. *Prolog-Type rule*

$$P(X_1, X_2, \ldots, X_n) \colon - Q_1(Y_{1,1}, Y_{1,2}, \ldots, Y_{1,n_1} \wedge \cdots \wedge Q_m(Y_{m,1}, Y_{m,2}, \ldots, Y_{m,n_m}).$$

To understand this formula some logic concepts are introduced. A predicate is defined by a set of variables and determines a function from the definition domain of its variables to $\{true, false\}$. A literal is a predicate or its negation. A clause is a set of predicates and a fact is a clause with only one literal that is true under any condition. Then, P is a predicate and it is called head of the rule. The expression on the right at right of the separator—is called body of the rule and is composed by a set of predicates (Q_i) joined by the logic connector 'and' ($\wedge$). The rule complies with the following restrictions:

(i) Every variable in the head predicate appears at least once in the body of the rule, and
(ii) Every variable in the body of the rule appears either in the head predicate and/or in another predicate of the body.

A rule is used to calculate the set of facts that makes true the head predicate.

2.5. Logical model for imprecise data

The Logic Database Model previously defined is extended in different approaches [24,25,29] to deal with the imprecise data. One of them is the Extended FREDDI (EFREDDI) architecture.

2.5.1. Extended FREDDI

EFREDDI [1] is a combination of FREDDI capabilities (storing and executing logical rules in a RDBMS) and FIRST capabilities (representation and handling of fuzzy data in a RDBMS) made by Blanco et al. In this architecture there are two kinds of relations depending on whether the instance of the relation is explicitly stored or if it is calculated using logical rules:

- *Extensional relation*: Defined as pair of sets (R, r) where R is a set of attributes that declares the relation structure, and r is the a set of tuples defining the content of the relation. This concept is the same as defined in the RDBM.
- *Intensional relation*: Defined as pair of sets (R, I) where R is a set of attributes that declares the relation structure, and I is the a set of logical formulas (rules), called instance generator (IG). The IG is used to obtain the content of the relation. Any rule present in I must follow the format declared in Definition 5.

Both relation types are extended to represent fuzzy data:

Definition 6. *Extensional Fuzzy Relation* is a *GFR*.

Definition 7. *Intensional Fuzzy Relation* is a pair of sets $(\mathcal{H}, I)$ where:

- $\mathcal{H}$ (named head) is a *GFR*, *and*
- I (named instance generator) is a set of rules that every fact of the associated predicate has to verify. Any rule present in I must follow the format declared in Definition 8.

The Prolog-Type (Definition 5) rule is also extended to handle imprecise data:

Definition 8. *Generalized Rule for deduction with matching degree* is defined as

$$\tilde{P}(X_1, X_2, \ldots, X_n, \gamma_{\tilde{P}}) \colon - \tilde{Q}_1(Y_{1,1}, Y_{1,2}, \ldots, Y_{1,n_1}, Y_{\tilde{Q}_1}) \wedge \cdots \wedge \tilde{Q}_m(Y_{m,1}, Y_{m,2}, \ldots, Y_{m,n_m}, \gamma_{\tilde{Q}_m}) \wedge \Psi,$$

with

$$\Psi \equiv \wedge(X_i =_{\alpha_{i,j,k}} Y_{j,k}) \wedge (Y_{j,k} =_{\beta_{j,k,l,p}} Y_{l,p} \wedge \phi_{j,k,l,p}(\Theta^{\theta}_{j,k,l,p}(Y_{j,k}, Y_{l,p}), \gamma_{j,k,l,p}),$$

where

- the expression $\wedge A_i$ represents $A_1 \wedge A_2 \wedge \cdots \wedge A_n$;
- the predicate $\tilde{Q}_k(Y_{k,1}, Y_{k,2}, \ldots, Y_{k,n_k}, \gamma_{\tilde{Q}_k})$ with $k \in \{1, \ldots, m\}$ represents the classical predicate $Q_k(Y_{k,1}, Y_{k,2}, \ldots, Y_{k,n_k})$ plus a value $\gamma_{\tilde{Q}_k}$ called matching degree. If we consider the set of facts calculated for Q_k, it is possible to measure the "distance'' of the remaining facts from to the calculated ones. $\gamma_{\tilde{Q}_k}$ is the nearness measure of the whole facts set. Note that an exact fact implies $\gamma_{\tilde{Q}_k} = 1$;
- the formula Ψ represents the set of rule constraints composed by fuzzy links between variables and the user conditions, where:
  - conditions $(X_i =_{\alpha_{i,j,k}} Y_{j,k})$ refer to GFC $\Theta = (X_i, Y_{j,k}) \geqslant \alpha_{i,j,k}$ where $\alpha_{i,j,k}$ is the threshold for the condition to be fulfilled. It indicates whether two variables are more fuzzily linked (or fuzzily equal) than a threshold,
  - conditions $(Y_{j,k} =_{\beta_{j,k,l,p}} Y_{l,p})$ refer to GFC $\Theta = (Y_{j,k} Y_{l,p}) \geqslant \beta_{j,k,l,p}$ where $\beta_{j,k,l,p}$ is the threshold for the condition to be fulfilled. It indicates whether two variables are more fuzzily linked (or fuzzily equal) than a threshold,
  - functions $\phi_{j,k,l,p}(\Theta^{\theta}_{j,k,l,p}(Y_{j,k}, Y_{l,p}), \gamma_{j,k,l,p})$ represent the *ad hoc* condition the user puts in the rule in order to concrete its meaning. It determines if the satisfaction degree for the comparison is greater than a threshold. The function arguments are:
    - $\Theta^{\theta}_{j,k,l,p}(Y_{j,k}, Y_{l,p})$ is a GFC defined on the classical operator θ and
    - $\gamma_{j,k,l,p}$ is the minimum satisfaction degree to evaluate the comparison to true;
- the matching degree $\gamma_{\tilde{P}}$ refers to the matching degree between a fact and an exact fact calculated for the predicate P. The $\gamma_{\tilde{P}}$ value depends on the matching degree of every fact used for the calculation in the body of the rule and on the satisfaction degree of every condition appearing in the formula Ψ. So the expression for $\gamma_{\tilde{P}}$ is

$$\Phi = (\Theta = (X_i, Y_{j,k}) \geqslant \alpha_{i,j,k}, \ldots, \Theta = (Y_{j,k}, Y_{l,p}) \geqslant \beta_{j,k,l,p}, \ldots, \Theta^{\theta}_{j,k,l,p}(Y_{j,k}, Y_{l,p}), \ldots, \gamma_{\tilde{Q}_1}, \ldots, \gamma_{\tilde{Q}_m}).$$

  In our case Φ corresponds to the min T-Norm.

  Every variable in the head predicate appears at least once in the body of the rule

- Every variable in the body of the rule appears in the head predicate and/or other different predicate of the body of the rule.

The relational representation of generalized rules is initially described in [20] and later imposed in [1].

2.5.2. An extended implementation for FREDDI

The intensional relations, extensional relations and generalized rules in the EFREDDI architecture are managed by means of a new language named DFSQL [1]. This language was being in two steps. In the first one, a language called DSQL [2] was created as an extension of SQL but incorporating deductive capabilities. In the second step the DSQL was extended with the FSQL defined for FREDDI in order to incorporate fuzzy data handling.

The EFREDDI implementation compromises the following elements:

- *Data Definition Language* (*DDL*): It is an extension of the classical DDL of SQL language that supports the definition of fuzzy attributes, fuzzy data types, intensional tables, rules and predicates. Section 4.4.2 shows an example of syntax.
- *Data Manipulation Language* (*DML*): Again it is an extension of the classical DML of SQL language to manipulate data that supports fuzzy conditions, fuzzy constants, fuzzy comparators, fuzzy arithmetic operators, fuzzy aggregate functions, compatibility degrees and thresholds.

2.6. Deduction algorithms

Usually in a logical database the deduction process is carried out by two different mechanisms: Prolog [6] and Datalog [9].

2.6.1. Classical deduction algorithms

Two classical algorithms were proposed to perform deduction using rules:

(i) Top-down: Based on the Prolog engine. Also called "tuple oriented algorithm''. It is based on the idea of instantiation of the variables present in the predicates in order to obtain the fact set.
(ii) Bottom-up: Based in the Datalog engine. Also called "relation oriented algorithm''. The idea is to create a query that calculates iteratively the fact set.

Both algorithms can be extended to handle imprecise data. To do it, is necessary to propagate into the algorithm the matching degree and the queries used to access to the database relations. In [4] algorithm (i) is studied in depth and in [1] a comparison analysis between them is made. From the implementation point of view, (ii) is better than (i) because it requires less resources to obtain the same fact set. The resources are defined as structures to store partial results, number of context changes (the control goes from a predicate to another one) and queries to the database. For this reason (ii) was the deduction algorithm used in GDB. Let's see in detail the bottom-up algorithm extended for generalized rules.

2.6.2. Bottom-up deduction algorithm for generalized rules

The Bottom-up deduction algorithm applied on a predicate P gives as result a query that calculates the set facts for P. The Intensive Relations will be replaced by its expanded query, so the final query will have a sub-query for any intensive predicate. If definition of P includes a recursive predicate, it will be substituted by the associated query that includes P, that will be substituted again. In this case, the algorithm execution will stop when no changes are detected in the relation associated with P. This algorithm is extended by Blanco in [1] to support Generalized Rules.

As an example let's apply the algorithm to a predicate defined by the following Generalized Rule:

$$\tilde{P}(X, Y, \gamma_{\tilde{P}}) \colon -\tilde{Q}(X_Q, Z_Q, \gamma_{\tilde{Q}}) \wedge \tilde{R}(X_R, Z_R, \gamma_{\tilde{R}}) \wedge \geqslant(\Theta^{=}(X, X_Q), \alpha) \wedge \geqslant(\Theta^{=}(Y, Y_R), \alpha) \\ \wedge \geqslant(\Theta^{=}(Z_Q, Z_R), \beta),$$

with $\Lambda = \{Q(a, b), Q(c, d), Q(e, f), R(g, h), R(i, j)\}$.

$\tilde{P}$ has associated the query expressed in Relational Algebra [14] (Fig. 3):

$$(\tilde{p}, \tilde{P}) = (\mathcal{H} = \{X, Y, G\}, \mathcal{B} = \{\Pi_{q,X,r,Y,\Phi(\Theta^{=}(q,Y,r,X))}(\sigma_{\geqslant(\Theta^{=}(q,Y,r,X),\beta)}(\tilde{q} \times \tilde{r})\}.$$

3. Modifications to DFSQL server for GDB

GDB uses as base system the improvements made to EFREDDI but it needs additional capabilities in the DFSQL server: "Macro", "Trace" and "Storing".

3.1. "Macro" capability in the DFSQL server

In GDB it is usual to build predicates that incorporate ranges. For example: "*get the set of events generated* 30*s after time T*''. This predicate can be translated into classical logic as

$$P(I, X) \colon -E(I, X) \wedge (X > T) \wedge (X < T + 30),$$

where the variable I represents the event identification, X is the event time, T is a constant and E is the event predicate. The predicate $(X < T+30)$ cannot be expressed using the DFSQL language because it is not possible to use an operator $(+)$ in the right part of an operand $(<)$. In order to solve this problem the DFSQL language is modified allowing this type of structures but isolating the right part between brackets. This extension is called "Macro'' due to it is ignored during the DFSQL query translation to SQL and it is reconstructed before its execution. A special table is created to store the "macro" information in the Extended Catalog (Fig. 2). The previous predicate can be defined as

$$P(I, X, T) \colon -E(I, X) \wedge (X > T) \wedge (X < [T + 30]).$$

The constant T is added in the consequent predicate as variable. This variable will be fixed with the desired constant before the rule execution. This is the solution adopted to represent constants in the DFSQL language.

$(\tilde{q}, \tilde{Q})$	
X	**Y**
a	b
c	d
e	f

$(\tilde{r}, \tilde{R})$	
X	**Y**
g	h
i	J

$\tilde{q} \times \tilde{r}$			
q.X	**q.Y**	**r.X**	**r.Y**
a	b	g	h
a	b	i	J
c	d	g	h
c	d	i	j
e	f	g	h
e	f	i	j

$\sigma_{\geq(\Theta^{=}(q.Y,r.X),\beta)}(\tilde{q} \times \tilde{r})$			
q.X	**q.Y**	**r.X**	**r.Y**
a	b	g	h
a	b	i	J
c	d	g	h
c	d	i	j
e	f	g	h
e	f	i	j

$(\tilde{p}, \tilde{P})$=$\prod_{q.X,r.Y,\Phi(\Theta^{=}(q.Y,r.X))}(\sigma_{\geq(\Theta^{=}(q.Y,r.X),\beta)}(\tilde{q} \times \tilde{r})\}$		
X=q.X	**Y=r.Y**	**G=**$\Phi(\Theta^{=}(q.Y, r.X))$
a	h	$\Phi(\Theta^{=}(b, g))$
a	j	$\Phi(\Theta^{=}(b, i))$
c	h	$\Phi(\Theta^{=}(d, g))$
e	j	$\Phi(\Theta^{=}(f, i))$

Fig. 3. Bottom-up deduction algorithm example.

3.2. "Trace" capability in the DFSQL server

GDB has the possibility to check it all the steps followed to assign the MoQ using a human readable format. To do it, it is necessary to know which predicate generate what fact. The Datalog deduction algorithm does not care about the single tuples (facts), all the operations are related to tables. The result of Datalog is a SQL query built from the predicates definition. The DFSQL server translator is modified to add extra attributes in the Datalog query to remember the predicate identification that generated the fact set. In order to not disturb the Datalog algorithm, the attribute addition is performed when the final query is generated. As example, let's consider the following predicate definition:

(i) $P(X, Y) :- Q(X, Z) \wedge R(Z, Y)$,
(ii) $Q(X, Y) :- S(X, Y)$ and
(iii) $Q(X, Y) :- T(X, Y)$,

with $\Lambda = \{S(a, b), T(c, d), R(b, f), R(d, i)\}$ and predicates identification: $P1$, $Q1$ and $Q2$ for predicates (i), (ii) and (iii), respectively.

The Datalog algorithm will give as result a query that calculates the fact set for p, P). This query can be expressed in SQL (without matching degree attributes) as

```
select P.X, P.Y from (select Q.X, R.Y
                      from (select S.X X, S.Y Z from S
                          union
                              select T.X, T.Y from T) Q, R
                      where Q.Z=R.Z) P
```

Table 5
Datalog Algorithm example

(p, P)		
X	Y	TRACE
a	f	$P1(Q1)$
c	i	$P1(Q2)$

At this point, the DFSQL server must add the new attributes set that indicates the predicate used. Those new attributes have a fixed value that fits with the predicate identification. In SQL, for adding a fixed attribute, it is necessary to put into quotes the constant and add it to the attribute list in the select part. In the example the new attributes are renamed to `TRACE`. Note that binary operator `//` returns the left operand value plus the right operant value. This operator is used to join the `TRACE` attribute value of the intensive predicates (as $Q1$). So the modified SQL query is:

```
select P.X, P.Y,P.TRACE from (select Q.X,R.Y, ('P1('//Q.TRACE//')') TRACE
                              from (select S.X X, S.Y Z, 'Q1' TRACE from S
                                    union
                                    select T.X, T.Y,'Q2' TRACE from T) Q, R
                                where Q.Z = R.Z) P
```

and the final fact set (see Table 5).

The first tuple in the relation has the value '`a`' that comes from the application of the predicate with identification $Q1$. This information is stored in the `TRACE` attribute value. Note that in `TRACE` is stored the set of predicates used to calculate the fact and also the sequence of application ($P1(Q1)$ for the first tuple). So, for any fact, the predicate application sequence used to calculate it will be stored. Using this sequence is easy to build in a human readable format an Explanation of the rules set used to calculate a fact.

3.3. "Storing" capability in the DFSQL server

This capability was introduced to optimize the performance of the execution of the query that involves intensive predicates. If the same predicate appears 'n' times in a predicate definition, it will be replaced by its expansion query 'n' times and the same query will be execute 'n' times too. The idea is to store the result table of the first occurrence of each intensional table and use that table as replacement in the query substitution.

4. Rosetta and GIADA

Rosetta [5] spacecraft will be the first one to undertake the long-term exploration of a comet (67P/Churyumov-Gerasimenko [11]) at close quarters. It was launched in 2004 and will start operating in 2014. It comprises a large orbiter, which is designed to operate for a decade at large distances from the Sun, and a small lander. Each of these ones carries a large amount of scientific experiments designed to complete the most detailed study of a comet ever attempted. One of the Rosetta's instrument is GIADA [8,12,13] (Grain Impact Analyzer and Dust Accumulator) that will measure the number, mass, momentum, velocity distribution and flow of dust grains coming from the comet kernel.

GIADA is divided in three main subsystems:

- Two GDS (Grain Detection System): The grain detection is activated when it crosses a laser curtain generated by four lasers and the scattered light is received by two set of photo receivers (left and right).
- One IS (Impact Sensor): The grain arrives to an aluminium square (IS plate) that can measure the grain impact using five piezoelectric sensors.
- Five MBS (Micro-Balance Sensors): MBS are devices that accumulate grains and are capable to calculate the mass of the deposited dust towards five different directions (Fig. 4).

Fig. 5 represents the GIADA data flow from its acquisition to its processing. The comet data acquisition will be made by the Rosetta payloads (as GIADA) and then they will be sent to Earth in telemetry packets. Data will be received by

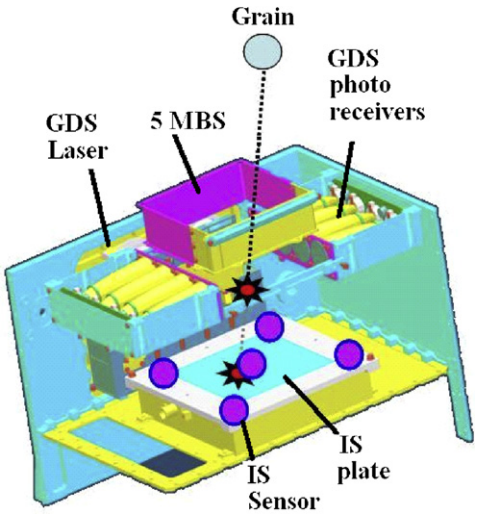

Fig. 4. GIADA subsystems.

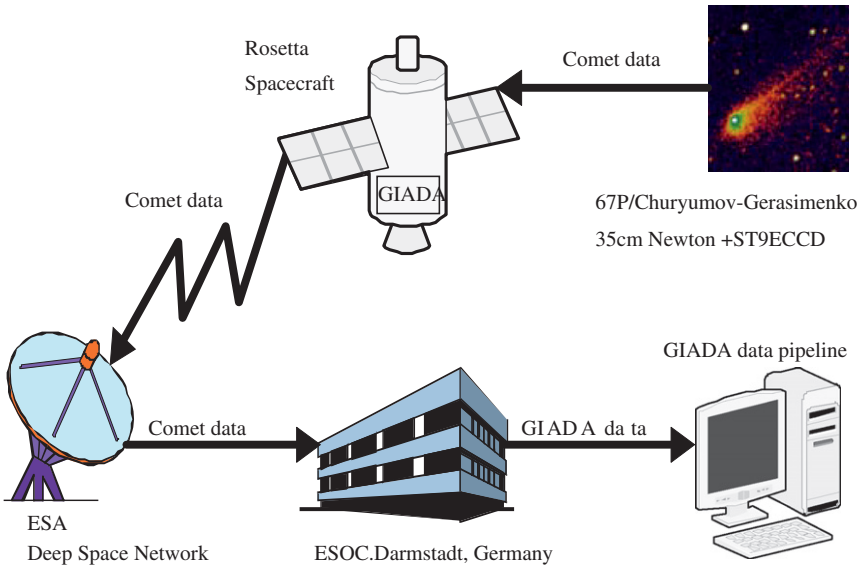

Fig. 5. GIADA data flow.

the European Space Agency (ESA) Deep Space Network and sent to the European Space Operations Centre (ESOC). Then, the whole data will be separated by instrument and re-sent to its specific data pipeline.

4.1. GIADA data problems

Data generated by GIADA will be processed in the GIADA data pipeline. Usually in the space instruments, the data pipelines are created when the instrument data are available. This gap implies a variable gap between instrument design and the data processing (10 years for GIADA). Data pipelines must work with unorganized data sets that sometimes are also meaningless, invalid, noisy or even not usable. Also, it is possible that data sets were generated in a not nominal situation (out of expected environment), so its validity must be studied and defined. In all these cases, the formal knowledge (technical documentation), and the background knowledge of the engineers that designed the instrument, is invaluable.

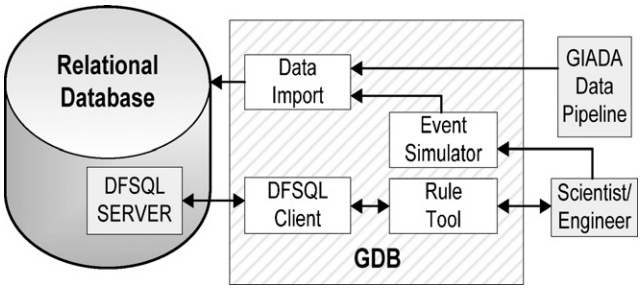

Fig. 6. GDB general working scheme.

Together with mentioned problems associated with the data pipeline, GIADA data have a set of obstacles inherent in any long term international space mission:

(1) *Long time running*: the gap between the instrument design and the data reception is about a decade, so human expert knowledge can be lost.
(2) *Geographical dispersion*: engineer teams are spread around the world, so knowledge about the instrument is also dispersed. ESA was involved in an "instrument video approach", a first solution to recover the knowledge.
(3) *Data cost*: space projects are extremely expensive, so every single data is important and cannot be ignored.
(4) *Pioneer data*: for the very first time, it will be possible to acquire in situ amounts of high quality data about comets so, previous data experience is not available to detect instrument errors.

4.2. The proposed tool

The main objective of the proposed application is to make easy the recovery of background knowledge (usually imprecise) of the engineers team and nominal knowledge stored in the formal documentation [21,22]. All this information will be used to perform a right interpretation of any GIADA instrument data assigning them a MoQ and obtaining and Explanation of how the MoQ was set. Knowledge manipulation will help to solve the problems in the data pipeline and also the problems (1) and (2) defined in the previous section. The MoQ + E will ease the solution for the problems (3) and (4).

GDB uses all theoretical and implementation results of GEFRED, FIRST, FREDDI and EFREDDI. GDB works as a client of a DFSQL Server (Fig. 2). DFSQL Server implements a deduction algorithm (see Section 2.6.2) based on Generalized Rules in order to obtain information that is not explicitly stored in the Database. Also it manages several data types (Table 1) that can be used in the rule or data definition, including imprecise information. The DFSQL Server has been extended to meet specific GDB requirements as "Macro'', "Trace" and "Storing'' (see Section 3). The MoQ + E is defined using those concepts. The MoQ calculation is really the matching degree defined for the Generalized Rules (Definition 8) and the Explanation is the "Trace'' capability.

The general GDB working scheme is shown in Fig. 6. Using the *Rule Tool* (RT), engineers/scientists can build, execute and debug their own rules or queries with real or "simulated data''. GDB implements a *DFSQL client* to connect with the Server (described in Section 2.3) and to require the query execution. Incoming data are managed by the *Data Import* and can have two main sources: "real data'' (*GIADA Data Pipeline*), or "simulated data'' (*Event Simulator*). Those data are directly stored in the RDBMS. The DFSQL server output is the MoQ + E assigned to every input data that will be sent to the client and then shown in the RT.

4.3. Main rule tool features

GDB has been built to facilitate the rule and query creation, debugging and execution. Fig. 7 shows a snapshot of GDB in a typical work session including a rule creation and its execution.

GDB allows navigation along the Relational Database Scheme using a field tree (2), selecting attributes (fields) that will be used as GDB output. Any field can be linked with other fields or predicates (8) using operators (6) to obtain a set of data restrictions (7). A rule tree (4) is also available to manage the stored rules previously created. The tool

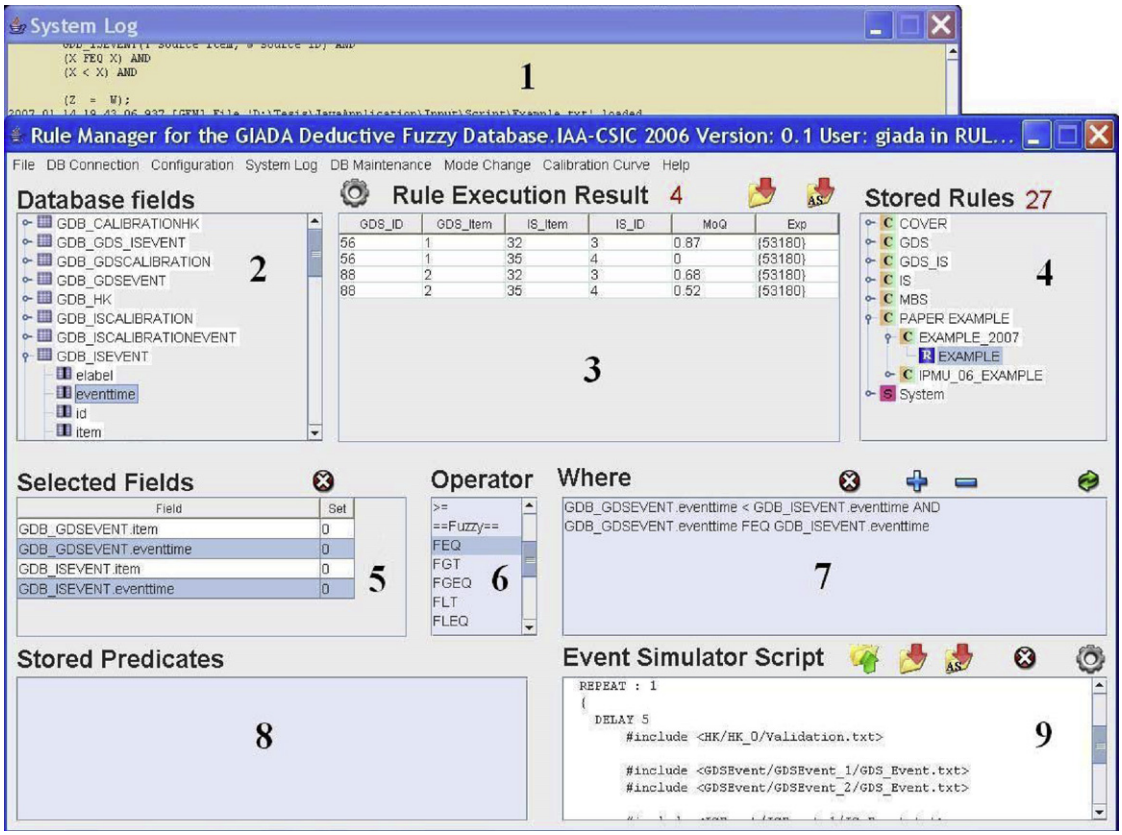

Fig. 7. Rule Tool snapshot.

connects with DFSQL Server in order to apply the rule set (4) to input data and generates the results (3). All the tool activities are logged (1) for analyzing step by step the GDB behavior. The typical process after a rule creation is its validation. This action includes the generation of a controlled data set using the Event Simulator (ES) (9). When rule data results (3) fit with expected ones then the rule is considered valid and its results can be checked with the well studied calibration data. Finally, these valid rules will process the "real'' data when available.

4.3.1. Assisted rule creation

The first step in the rule creation is to have a precise idea of the rule to be built. A rule template has been created for this purpose. With the template filled in, the user is able to start to work with the GDB. If there is any doubt or ambiguity during the rule creation, the user that wrote the template is contacted. Using the template, the rule builder must understand the rule purpose and find the set of fields or previously stored rules involved in it. Also the relationship between fields and operators to be used (fuzzy or not) must be clear in the template.

At this point the rule must be stored in the rule base. The rule is translated into the DFSQL Server language in order to save it. This translation is transparent to the user and can be checked in the detailed rule description. This translation involves accesses to fuzzy and classical catalogue and also an automatic join procedure. Without this procedure, the user would have to declare how the tables must be linked. If the rule uses more than 2 or 3 tables this task could be difficult. The procedure of auto-join solves the table link but only can be applied on Relational Databases with a hierarchical scheme (Fig. 6) where only exist one father table (*GDB_Packet*). There is also an automatic name assignment for tables and variables (required by DFSQL Server syntax) that is hidden to the user.

When the rule has been saved it is necessary to debug/test it using controlled data. To do it, GDB incorporates the Event Simulator (ES). Finally, after the rule application, results can be revised including the MoQ and identifications of rules used to assign that MoQ to any data (the explanation). If MoQ does not fit with the expected data used in the ES, it is time to review those rules involved in the explanation. Results can be saved in a plain ASCII file that can be

used as input to other rules. Input data are not modified at all because the MoQ + E are added as extra columns to the resulting table.

4.3.2. Rule storage

Rules are stored in folders (categories), and those folders can store also other folders (see Fig. 7). Any change in the rule (rule source, name, field, . . .) is logged. So it is possible to reconstruct all the rule history from its creation to the current state. This is a powerful tool to see the rule evolution and find possible bugs in the rule definition or implementation.

There are two types in the rule storage process, "the private user rule storage'' (all categories except the *system* one) and "the user's shared rule storage'' (*system category*). In the "private storage type'', the user can play with rules (create, debug, delete. . .) without disturbing the other users because the "private storage'' is not visible by them. The "shared storage type'' is composed by rules that have been passed an exhaustive rule validation and there is no doubt about their effectiveness. So, the set of rules involved in any GDB execution will be private user rules plus *system* category rules. With this configuration it is possible that the user can work with provisional rules before they become "official'' ones.

4.3.3. Event simulator

The ES accepts ASCII scripts files (describing the data to be generated) as input and produces nominal Rosetta data packets (Fig. 5) as output. A customized language has been created to work with GIADA data. The customization allows that several input data (usually with not relevant information) can be declared as "calculated'' leaving its final value to the GDB, making the script creation easy. An ES script can set any data field defined in the Rosetta packet format [22]. This means that it is possible to build data packets storing controlled data. Usually, the only way to get controlled data is:

- inserting manually the raw data into the tables and
- using hardware event generators.

In the first case, it is difficult to preserve the integrity of the Relational Scheme when data are inserted in the corresponding tables and also it is not easy to generate large sets of data manually. In the second case, hardware event generators can produce large sets of data but the experience indicates that it is not simple to have precision either in data values nor in data types (e.g. IS or GDS + IS events). The ES solves these problems because data and their types are set by the user in ASCII files. Loop sentences included in the ES script language generate large sets of data and the insertion in the Database is automatic.

4.3.4. Other features

GDB has been developed in Java ©, so it can run with several operating systems (Sun, Linux, Windows, Macintosh. . .) and processors (Intel, Power Macintosh, AMD, Alpha. . .). For this reason, usually the user does not need a special hardware and/or software (excluding the Java Virtual Machine to run the GDB) and the GDB developer does not maintain different versions.

DFSQL server has a dedicated computer running Oracle © 64 bits server on Fedora 7 64 bits edition. In the hardware side, the computer has a dual Xeon processor with 4 GB RAM memory and 2 SATA (350 GB) hard disks in RAID 1 configuration. DFSQL Server allows GDB multi-client access. For international teams located in different countries, it is a good solution to share the data resources. It is foreseen to build a webpage describing the GDB including the possibility to download it and test it with a set of input data.

GDB has two execution modes, the *Rule Mode* (described in this text) and the *SQL Mode*. In the latter mode, the user can build (and execute) visually SQL sentences without any knowledge about SQL syntax and it has equivalent features than the existing in the Rule Mode.

4.4. Rule example

In this section a rule example is introduced from its definition until its execution. Using an expert knowledge piece, a rule is created. Then it will be translated into the DFSQL language and finally it will be execute using controlled data. This is the traditional way to work with rules. The same process will be repeated but this time using GDB. We will

```
TM Packet Received from GIADA:

Source Sequence Count: 0

Packet Length: 79

Packet Type, Subtype: 3, 25

-----------------------------------------------

0D A4 C0 00 00 4F 00 01 11 B0 06 00 40 03 19 00

00 01 EE EE 00 00 00 20 05 D0 00 00 00 00 00 00

00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

0F FF 00 00 00 00 00 00 00 00 00 00 00 00 00 00

00 00 00 00 08 A4 01 B5 00 9F 02 C2 00 00 00 00

00 00 00 00 8D 2F

-----------------------------------------------
```

Fig. 8. Example of GIADA housekeeping telemetry.

show how to do it in only five steps, simplifying the work and avoiding the DFSQL syntax. The GIADA Relational Database Scheme that will be used in the rule example is introduced.

4.4.1. GDB Relational Database Scheme

The nominal input raw datum generated by GIADA is expressed as a sequence of digital counts (DC) in hexadecimal values ([22] and [21]). Those DC come from the measure of physical phenomena (temperature, current…) using sensors. Later, DC will be converted to physical magnitudes (amps, centigrade degrees, volts…) in the data pipeline using a set of calibration curves. All GIADA data are stored in the Rosetta telemetry packet format and sent to Earth (Fig. 5). Fig. 8 shows a nominal GIADA Housekeeping (HK) telemetry packet implementing the Rosetta format: first 16 bytes (first line) for *Packet Header* + *Data Field Header* set of fields. Remaining data are devoted to the *Source Data* (GIADA data) keeping the last 2 bytes for a Cyclic Redundancy Checking in order to detect data corruption. An ASCII header precedes the hexadecimal data to allow an easy reading by the human being.

There are two main GIADA telemetry packet types (Fig. 5):

- *Housekeeping packets* (table *GDB_HK*): This type of packet shows the status and health of the instrument periodically (by default in 40 s intervals). This is the information that will be used to obtain the data acquisition environment. Usually the HK is composed by a set of readings of temperatures, currents and voltage sensors plus instrument mode, patches status, etc…
- *Science packets*: One per GIADA subsystems (Fig. 4). Any subsystem has its own calibration packet (to check its health). The instrument is capable to detect a specific event that will be stored in a *science packet*. Every *science packet* has associated one table in the Relational Database Scheme (Fig. 9):
  - GDS: table *GDB_GDSEvent*;
  - IS: table *GDB_ISEvent*; and
  - MBS: table *GDB_MBReadingEvent*.

When the GDS and the IS subsystems detect a grain into a predefined time range, the two independent events build a composed GDS + IS event (table *GDB_GDS_ISEvent*). It is possible to group a set of events (GDS, IS, MBS, GDS + IS) in a single one science packet in order to reduce the header overhead. The event set shares the same acquisition environment (table *GDB_ScienceHK*). The field *Item* identifies the event storing order inside the event set. The table *GDB_Packet* stores in the field *ID* the instant (microsecond) in which a packet was stored in the Database.

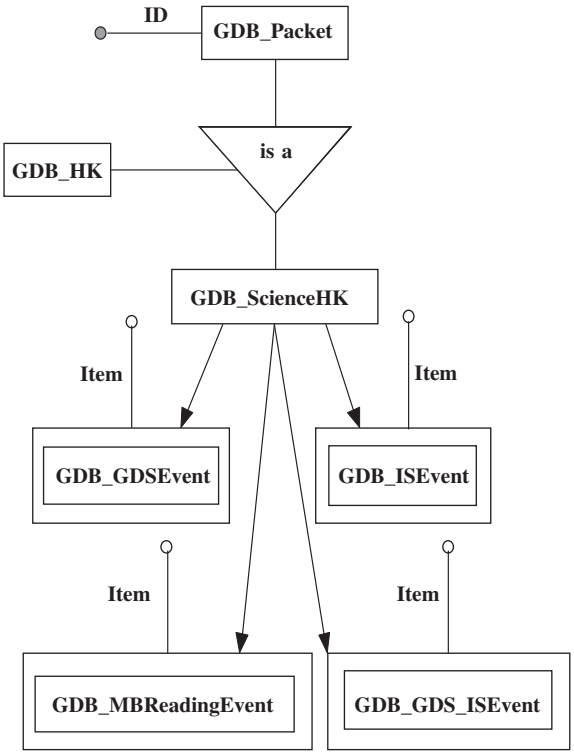

Fig. 9. Excerpt of GDB Relational Database Scheme.

The final Database Relational Scheme is a hierarchical tree with only one root (the table *GDB_Packet*). With this type of structure it is possible to link all tables in only one way: following the foreign keys to calculate the table natural joins. GDB will use this property giving the user a transparent table association making easy the query building process. The scheme is shown in Fig. 9 using the Entity–Relationship model. For sake of simplicity only the fields that compose the primary key of the tables have been detailed and certain tree branches (Calibration, MB_Heating, Calibration Curve and Session) are omitted.

4.4.2. Rule creation without GDB support

As an example we will consider the following simplified knowledge extracted from the formal documentation [22]: "Two independent GDS and IS events are considered a composed GDS + IS event if the flight time (time to cover the 10 cm distance between GDS and IS) is less than 327 ms. In other case the events are not considered as a pair''.

The current model about comet grains velocity indicates that this value of 327 ms to cover 10 cm is a reasonable minimum velocity. But these theories must be contrasted with real measures. The current data about comets comes from indirect observation of real comets or from mathematical models of behavior, so this velocity estimation could be imprecise. A composed event has more scientific relevance than independent events because it usually reflects complete grain detection. So, it is interesting to know how the GDS and IS events could be matched around this 327 ms limit.

This is a typical fuzzy query that can be translated into a rule and processed by the DFSQL Server. The GDS and IS events (tables *GDB_GDS_Event* and *GDB_IS_Event*, respectively), have its own detection timestamp called *eventTime*. We need to compute the similarity of two *eventTime* values with a margin around of 327 ms, for example 350 ms. To do that, the DFSQL operator fuzzy equality (*FEQ*) will be used. Taking into account the fuzzy types defined previously, the most appropriate type for the *eventTime* attributes is the Fuzzy Type 1 due to it supports crisp representation (the *eventTime* data) and fuzzy query (in particular *FEQ*). The *eventTime* attribute must be defined as fuzzy attribute in the *GDB_GDS_Event* and *GDB_IS_Event* tables:

- *eventTime FTYPE1*(350,500) *NUMBER*(15) *NOT NULL*.

The number 350 is the *margin* (*Fig.* 1) and number 500 is the *much*.

Table 6
GDB_GDSEvent

ID	Item	EventTime
56	1	3433
88	2	3700

Table 7
GDB_ISEvent

ID	Item	EventTime
32	3	3500
35	4	4030

The other table attributes are defined in the classical SQL syntax. Both tables must be created using the FSQL (included in the DFSQL Server) in order to process the definition of fuzzy attributes.

The intensional table associated with the rule must be defined first. This table will store the pairs of events found by the rule:

- *create intensional table GDB_INT_EXAMPLE* (*GDS_ID number*(16), *GDS_Item number*(2), *IS_ID number*(16), *IS_Item number*(2)).

Attributes *GDS_Item* and *IS_Item* are necessary because they are part of the primary key (Fig. 9, tables *GDB_GDS_Event* and *GDB_IS_Event*). Now, the rule that stores the knowledge previously introduced is defined:

- *create rule for GDB_INT_EXAMPLE* (X, Y, Z, W) *as GDB_GDSEvent* (*X source ID*, *Y source item*, *Q source eventTime*) *and GDB_ISEvent*(*Z source ID*, *W source item*, *T source eventTime*) *and* $(Q < T)$ *and* $(Q\ FEQ\ T)$;

where:

- *GDB_INT_EXAMPLE* is the consequent predicate and also the rule name. It has associated the intensional table previously defined. It has four variables (X, Y, Z, W) that match with the intensional table attributes.
- *GDB_GDSEvent* and *GDB_ISEvent* are antecedent predicates. Both predicates have homonymous associated tables. The two first variables match with the two first consequent predicate variables. The last ones will be used in predicate matching. Every variable must have a data source in the associated table.
- $(Q < T)$ is an antecedent comparison for pruning those IS events that has been detected before a GDS event. This is a classical predicate.
- $(Q\ FEQ\ T)$ is an antecedent predicate. This predicate represents the GFC $\Theta^{\theta}(Q, T)$ defined on the EFC $\theta = FEQ$. The threshold γ for this predicate is set to zero because all the facts must be returned (including those with satisfaction degree zero). This predicate will measure the fuzzy equality (as described in Section 2.2.2) as the intersection area of two possibility distributions associated to the Q and T values.

Some simplifications are adopted from the Generalized Rule (Definition 8). Linked variables between predicates use the classical matching $(X_i =_1 Y_{j,k})$ $(Y_{j,k} =_1 Y_{l,p})$, so Ψ is defined as $\Psi \equiv \phi(\Theta^{\theta}(Q, T), 0)$. Finally, the rule matching degree is $\Phi = (\Theta^{\theta}(Q, T))$. The MoQ (in fact the matching degree Φ) is calculated by DFSQL Server storing it as an extra attribute (*MoQ*) in the table *GDB_INT*_EXAMPLE (Table 8). The Explanation is also an extra attribute (*Exp*) in the same table and its value is the set of rule identification that modify the tuple content (see Section 3).

Below, controlled data (inserted with the ES) used to check the rule are introduced in Tables 6 and 7.

Running DFSQL Server with the rule created and the set of controlled data, we obtain the following results as shown in Table 8.

In the example, the events pair with items (GDS_Item = 1, IS_Item = 3) has a high MoQ (0.87) due to its detection interval (67 ms) is inside the nominal margin defined (350 ms), so this pair is a good candidate to be managed as composed GDS + IS event. The pair (2, 3) has a MoQ = 0 because the predicate $(Q = 3700 < T = 3500)$ is

Table 8
GDB_INT_EXAMPLE

GDS_ID	GDS_Item	IS_ID	IS_Item	MoQ	Exp
56	1	32	3	0.87	{53180}
56	1	35	4	0	{53180}
88	2	32	3	0	{53180}
88	2	35	4	0.52	{53180}

evaluated to false. The pair (1, 4) with a detection interval of 597 ms has MoQ = 0 because is greater than the defined *much* (500 ms). And finally, the pair (2, 4) with detection interval of 330 ms has a MoQ greater than 0.52, so this pair could be considered as compose event. Note that the (2, 4) pair will never be considered as composed using the formal limit of 327 ms and the tool, not only discovers the possible association, but also measures the quality of it. The DFSQL Server has associated the identification '531810' to the rule. So the GDB "Trace'' capability (see Section 3) sets this value in the *Exp* attribute because only this rule has been executed.

It is important to know that the input data are not modified at all even if the rule set gives the worst possible quality. This implies that the original and the calculated data can be used as input for a more specialized tool in the data pipeline that may use the MoQ as input data filter.

4.4.3. Rule creation with GDB support

Only five steps are needed to create and get the results of a rule using the GDB support:

Step 1: Selection of fields (Fig. 7, numbers 2 and 5) using the tree field.
Step 2: Selection of operators and apply them to fields in order to constrain data (Fig. 7, number 6 and 7).
Step 3: Rule saving (Fig. 7, number 4).
Step 4: ES script build (Fig. 7, number 9). It is necessary to write a script that generates the set of controlled data that will be used as a rule input.
Step 5: Obtaining results (Fig. 7, number 3).

Note that it is not necessary to use any sentence of the DFSQL Language. Also, the table editing to set the input data is transparent using the ES. Reviewing the result table is easy to discover whether the rule was well designed. It is only necessary to check the MoQ + E and compare it with the expected values calculated from the controlled data.

4.5. Conclusion and future work

Scientific instrument knowledge is the key to perform a reliable data interpretation. GDB is introduced to recover this knowledge and to apply it, obtaining a Measure of Quality (MoQ) of the data to be processed. GDB is based on previous theoretical and implementation results in the field of Fuzzy Relational Deductive Database Systems (FRDDS). GDB handles fuzzy data, for representing and querying imprecise information of the instrument sensors, and rules, for managing the knowledge. GDB helps to solve data problems associated with data pipelines and with a pioneer long term international space mission. In addition to the MoQ, GDB offers the option to check all the steps ("Explanation'' GDB capability) followed in the MoQ assignment using a human readable format. Using GDB, knowledge about the instrument is compiled, organized and automated to process raw data allowing experts to get a better assessment of data. Using GDB, the expert is capable of modify the set of rules, so it can be adapted to the mission evolution. All this work is done using a visual interface, avoiding the user to know all the syntax associated with the FRDDS.

GDB is currently under development, so, simple results have been achieved and only with simulated data. A rule set has been tested to measure the quality of pairs of events initially not associated. Also, GDB was used to simulate sensor data readings in no nominal conditions, preparing a rule set that can handle them in order to obtain useful information. Valuable results will be obtained in 2014 when the real GIADA data arrives to Earth and can be processed with the knowledge collected at that moment.

After a deep GDB validation process, it is expected that it will be included in the instrument pipeline. GDB output (data and MoQ + E) will be used as input to other specialized tools where the MoQ are invaluable information for concentrating efforts and the research resources. Finally, it is possible to use data mining techniques to discover, in an

automated or guided way, relationships between instrument parameters that are unknown or poorly documented. This process generates a feed-back with new rules that can be added to the initial rule set.

Acknowledgements

This work has been supported by the following research projects of the Spanish government:

- ESP-2003-00357 and
- TIC2003-08687-C02-2.

References

[1] I. Blanco, Deducción en Base de Datos Relacionales Difusas, Ph.D., Department of Computer Science and Artificial Intelligence, University of Granada, Spain, 2001.
[2] I. Blanco, J.C. Cubero, O. Pons, M.A. Vila, An implementation for fuzzy deductive relational databases the recent issues on fuzzy databases, in: G. Bordana, G. Passi (Eds.), Studies in Fuzziness and Soft Computing Series, Physica-Verlag, Wurzburg, 2000, pp. 183–207.
[3] I. Blanco, N. Marín, O. Pons, M.A. Vila, An extension of data description language (DDL) for fuzzy data handling, in: H. Larsen, J. Kacprzyk, S. Zadrozny, T. Andreasen, H. Christiansen (Eds.), Flexible Query Answering Systems, Recent Advances, Advances in Soft Computing Series, Physica-Verlag, Wurzburg, 2000, pp. 54–64.
[4] I. Blanco, M.J. Martín-Bautista, O. Pons, M.A. Vila, A tuple oriented algorithm for deduction in a fuzzy relational database, Internat. J. Uncertainty Fuzziness Knowledge Base Systems (2003) 47–66.
[5] P. Bond, Rosetta's Comet Chase, ESA Brochure Series, Vol. BR-179, 2001.
[6] I. Bratko, Prolog, Programming for Artificial Intelligence, second ed., Addison-Wesley, Reading, MA, 1990.
[7] B.P. Buckles, F.E. Petry, A fuzzy representation of data for relational databases, Fuzzy Sets and Systems 7 (1982) 213–226.
[8] E. Bussoletti, J. Rodríguez, J.J. López, The GIADA experiment for Rosetta mission to comet 46P/Wirtanen: design and performances, Adv. Space Res. 24 (1999) 1139.
[9] S. Ceri, G. Gottlob, L. Tanca, Logic Programming and Databases, Surveys in Computer Science, Springer, Berlin, 1990.
[10] E.F. Codd, A relational model of data for large shared data banks, Commun. ACM 13 (6) (1970) 377–387.
[11] L. Colangeli, E. Epifani Mazzotta, P. Palumbo, The New ROSETTA Targets, Vol. XI, Springer, Berlin, ISBN: 1-4020-2572-6, 2004.
[12] L. Colangeli, R. Morales J. Rodríguez, J.J. López, GIADA: Grain Impact Analyser and Dust Accumulator, ESA SP 1165, 2000.
[13] L. Colangeli, R. Morales, J. Rodríguez, J.J. López, GIADA for the ROSETTA space mission: performances and calibration, Geophys. Res. Abst. 5 (2003) 09230.
[14] C.J. Date, An Introduction to Database Systems, Vols. I and II, Addison-Wesley, Reading, MA, 1990.
[15] H. Galiere, J. Minker, J.M. Nicholas, Logic and databases: a deductive approach, Comput. Surveys 16 (2) (1984) 153–185.
[16] J. Galindo, Tratamiento de la imprecisión en bases de datos relacionales: Extensión del modelo y adaptación de los SGDB actuales, Ph.D., Departament of Computer Science and Artificial Intelligence, University of Granada, Spain, 1999.
[17] J. Galindo, J.M. Medina, O. Pons, J.C. Cubero, A server for fuzzy sql queries, in: Proceedings of the Third International Conference on Flexible Query Answering Systems, 1998, pp. 164–174.
[18] J. Galindo, J.M. Medina, A. Vila, O. Pons, Fuzzy comparators for flexible queries to databases, in: Iberoamerican Conference on Artificial Intelligence, IBERAMIA'98, 1998, pp. 29–41.
[19] J.M. Medina, O. Pons, J.C. Cubero, M.A. Vila, GEFRED: a generalized model of fuzzy relational database, Inform. Sci. 76 (1994) 87–109.
[20] J.M. Medina, O. Pons, M.A. Vila, FREDDI: a fuzzy relational deductive database interface, Internat. J. Intell. Systems (1997) 597–613.
[21] R. Morales, GIADA-2 Software User Manual, RO-GIA-IAA-MA-009, Issue 2, ESA, April 2001.
[22] R.M.M. Morales, I. Olivares, GIADA Software Interface Control Document, RO-EST-RS-3009/EID B, Issue 2, ESA, May 2001.
[23] J. Mosterin, Lógica de primer órden, ed., Ariel S.A., Barcelona, 1983.
[24] O. Pons, Representación lógica de bases de datos difusas. Fundamentos teóricos e implementación, Ph.D., Departament of Computer Science and Artificial Intelligence, University of Granada, Spain, 1994.
[25] O. Pons, M.A. Vila, J.M. Medina, Handling imprecise medical information in the framework of logic fuzzy databases, Fuzzy Systems & A. I., Vol. 3(1), Ed. Academiei Romane, 1994, pp. 5–25.
[26] H. Prade, C. Testemale, Generalizing database relational algebra for the treatment of incomplete/uncertain information and vague queries, Inform. Sci. 34 (1984) 115–143.
[27] S. Shenoi, A. Melton, An extended version of the fuzzy relational database model, Inform. Sci. 52 (1990) 35–52.
[28] M. Umano, Freedom—0: A fuzzy database system, in: M. Gupta, E. Sanchez (Eds.), Fuzzy Information and Decision Processes, North-Holland, Amsterdam, 1982.
[29] M.A. Vila, J.C. Cubero, J.M. Medina, O. Pons, Some recent advances in fuzzy relational and fuzzy deductive databases, European Research Consortium for Informatics and Mathematics, Barcelona, Spain, November 1994, pp. 161–176.
[30] L.A. Zadeh, Fuzzy sets, Inform. Control 8 (1995) 338–353.
[31] M. Zemankova, A. Kandel, Fuzzy Relational Data Bases—A Key to Expert Systems, Verlag TUV, Rheinland, 1984.